# SELF-X: Loss-aware SDN control with episodic memory

*Measured revision 4 | Editable project report | 20 October 2026*

**Course: EC466: Software Defined Networks | Team: SDN_07 | Semester: Autumn 2026**
**Institution: Indian Institute of Information Technology Dharwad (IIIT Dharwad)**
Team Members: M Jagadeeswar | J Ganesh Kumar Reddy | Harsh Raj

## Abstract

SELF-X studies an SDN control loop that observes traffic, retrieves its own earlier state-action-outcome episodes, proposes a bounded intervention, validates the rule and learns from measured feedback. The local website provides replay, comparisons, provider provenance and submission files. This revision addresses avoidable meter loss and unbounded provider waits, and adds an end-to-end simulated rule lifecycle demonstration.

The fresh primary suite contains 86,400 controller windows: eight traffic scenarios, ten paired held-out seeds, 120 windows and nine controllers. A4 mean total loss was 7.075% versus H1 7.391%; reward was 0.082764 versus 0.077639. Modeled delay increased from 17.131 to 17.452 ms. Loss includes deliberate meter drops. These results establish a loss/reward improvement over H1 with a delay tradeoff, not uniform dominance.

H2 uses the same capacity-aware meter without memory. Its reward was 0.083304, compared with A4 0.082764. A4's bounded exploration remains in the results; the primary suite does not establish an extra reward benefit from memory over H2. Gains over H1 must therefore be attributed partly to better rate rules.

The final gemini pilot used actual authenticated requests on two fresh seeds, with five-second observation arrivals and 24 windows per mode. Previous/bounded mean full-cycle time was 0.486/0.487 s; P95 1.087/1.511 s. Five-second misses were 0/0. Bounded requests answered 10/10, with 0 local timeouts and 0 admission deferrals. Fallbacks remain explicit.

The final openrouter pilot used actual authenticated requests on two fresh seeds, with five-second observation arrivals and 24 windows per mode. Previous/bounded mean full-cycle time was 0.524/1.222 s; P95 2.094/3.620 s. Five-second misses were 0/0. Bounded requests answered 5/10, with 5 local timeouts and 0 admission deferrals. Fallbacks remain explicit.

All network outcomes are generated by a finite-buffer fluid model. API requests are real; rule installation occurs only in a simulated OpenFlow table. No physical switch, packet testbed, production SLA or general LLM-quality improvement is claimed.

# 1 Project objective and architecture

The project goal is to reduce repeated congestion by reusing the controller's own experience while keeping rule changes bounded and auditable. The website is a local experiment and review interface, not a public hosting service. The validated Windows execution path is the portable simulator in this folder.

| Stage | Implementation | Reviewable evidence |
|---|---|---|
| Observe | Seeded demand plus persistent queues | Traffic JSON, aggregate flow CSV |
| Retrieve | Capacity-scaled contexts and parameter-conditioned episodes | Prior cycle IDs and evidence margins |
| Suggest | Gemini / free OpenRouter, local memory or exact cache | Raw model proposal and origin |
| Validate | Known target, exact flow, route and rate bounds | Guard result and rejected proposal |
| Install | Simulated OpenFlow ADD, output and optional meter | Cookie, instructions, hard timeout |
| Measure and reflect | Fluid delivery, delay, loss and reward | Conserved volumes and local reflection |

Read README.md first, then docs/PROJECT_GUIDE.md, docs/BENCHMARKS_V4.md and docs/RULE_DEMO.md. docs/ARCHITECTURE.md and ARCHITECTURE.Rmd map source modules, entry points, persisted data and UI routes. All requested code, data and deliverables remain inside selfx.

The original Linux Mininet/Ryu adapter is retained. It is separate from this Windows fluid-model validation and needs an independent real-switch or emulator test before deployment claims.

# 2 Simulated traffic and network accounting

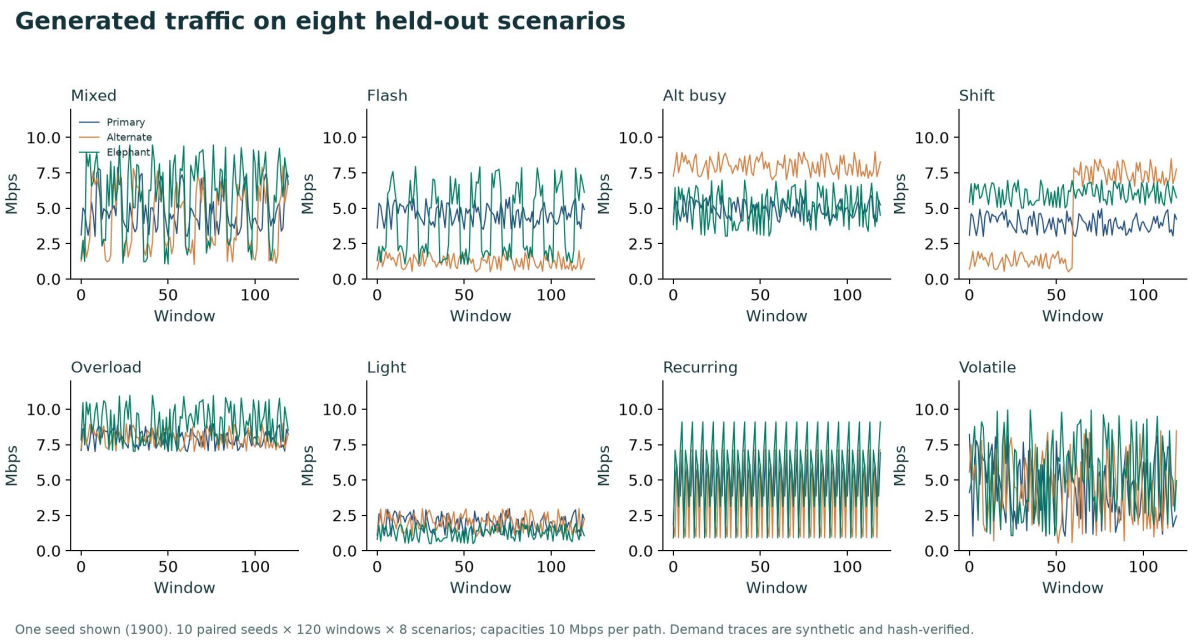



**Figure 1. Generated demand, one held-out seed shown.**

Each path has capacity 10 Mbps, a 0.5 Mbit finite queue and 8 ms propagation delay. Each window lasts five seconds. One elephant flow starts on the primary path; background traffic occupies both paths. Queues persist between windows, and each condition starts with empty queues and memory.

The eight families cover mixed demand, flash crowds, alternate-path pressure, a distribution shift, persistent dual overload, light load, repeated contexts and unpredictable demand. Every controller receives the same demand trace within a seed. The 9,600 unique demand windows are exported as a NetFlow-style aggregate table; it is not captured packets or a wire-format IPFIX message.

Accounting conserves volume: prior queues + offered traffic = delivered traffic + meter drops + overflow + final queues. Average queue occupancy is integrated over each constant-rate window; delay is a delivery-weighted fluid proxy. Total loss includes meter drops. P95 network delay means a percentile of window means, not individual packet RTT.

# 3 Parameter-aware self-embedding memory

| Component | Previous v3 A4 | Current A4 |
|---|---|---|
| Episode key | Context and action label | Context, action and actual meter parameter |
| Meter | Fixed 80% of elephant demand | Capacity fitting, bounded headroom candidates |
| Retention | 96 episodes, four per cell | 128 episodes, six per action/parameter/context cell |
| Context | Capacity-scaled demand, margins, queues | Same transparent numeric context |
| Support | Three matching prior episodes | Three prior observations per parameter |
| Choice | Conservative action advantage gate | Explore bounded caps, then compare reward evidence |
| Persistence | Batched atomic JSON save | Batched atomic JSON save; cached context vectors |

The memory uses hand-designed capacity-scaled features rather than a trained neural embedding. It preserves absolute load, congestion/feasibility margins and both queues. Retrieval requires the same feasibility regime and a local magnitude caliper. Nearby recent observations receive higher weight; only cycle numbers strictly earlier than the current cycle are eligible.

For an unsafe-to-reroute congested primary, A4 explores headroom values 0, 0.1 and 0.3 Mbps. Equivalent caps after the safety floor are one candidate. Each candidate needs three local observations before supported selection. Up to six neighbors contribute a distance/recency-weighted reward and spread margin. This evidence margin is a decision heuristic, not a calibrated confidence interval.

No candidate simulator outcome is evaluated during A4 selection. Observed reward and current-state information train the local episode store after the action. The privileged O2 reference evaluates current candidates explicitly and remains separate. A separate 120-window replay records 15 exploration windows, 45 supported memory decisions and 60 local rules, illustrating the full cold-start-to-reuse mechanism.

# 4 Smaller rate changes and unchanged safety bounds

The old rule removes 20% of an elephant flow whenever rerouting is unavailable. That can discard more traffic than the capacity excess requires. The revised meter admits the larger of 80% of elephant demand, the residual primary capacity minus a small headroom, and 512 kbps; it never admits more than offered elephant demand. The cap rounds upward to an integer kbps to preserve the 20% safety bound. A numerical boundary repair admits one extra kbps when binary arithmetic would otherwise falsely reject an exact 20% reduction; the strict guard is unchanged.

Example: primary background 5 Mbps, elephant 5.2 Mbps and busy alternate background 8 Mbps. H1 meters the elephant at 4.16 Mbps. With 0.1 Mbps headroom, the revised cap is 4.9 Mbps: only the capacity excess plus headroom is removed. The independently validated target is switch 1, in_port 1, IPv4 destination 10.0.0.8; rerouting uses output port 3.

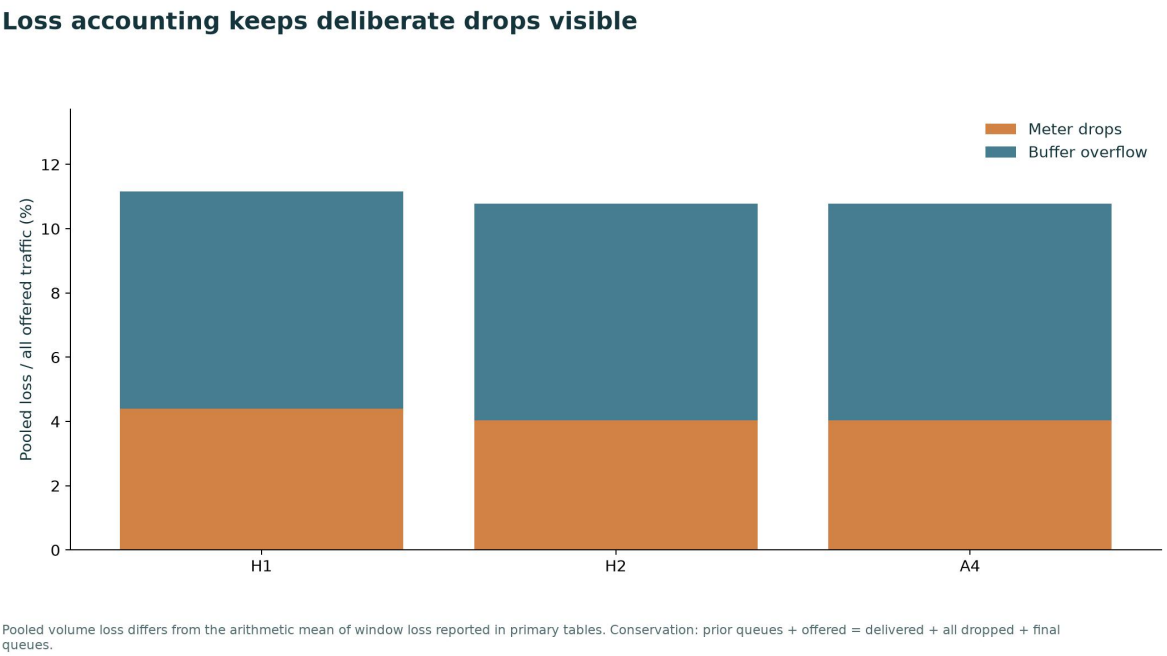



**Figure 2. Pooled offered-volume loss separates deliberate drops from overflow.**

Safety limits stay unchanged: rerouting must keep alternate demand at or below 90% of capacity; a meter may remove at most 20% of the target flow and must be at least 512 kbps. Less avoidable loss does not remove the physical overload lower bound; heavily congested traffic still loses volume.

# 5 Provider deadline and efficient local reuse

The former adapter could wait for pacing plus a 25-second transport timeout. The new control path isolates a compact provider call in a cancellable local process. One idle worker reuses its pooled HTTPS session after completed calls; startup is charged to its first request. A 3.8-second total wait budget includes admission pacing, process startup, IPC and cancellation; cleanup leaves time for safety validation, rule accounting and memory update before the five-second target.

| Origin | What happened | Fresh API answer? |
|---|---|---|
| api | Valid fresh proposal received and locally validated | Yes |
| api_cache | Exact context reused a validated API action | No |
| local_rule_cache | Exact context reused a validated local fallback rule | No |
| healthy_bypass | Current primary demand fits capacity | No |
| episodic_memory | Supported prior parameter evidence selects the rule | No |
| deadline_fallback | Local wait budget expired; worker terminated and joined | No |
| provider_fallback | Provider unavailable or admission deferred | No |
| validation_fallback | Fresh proposal failed the independent safety gate | Answer received; rejected |

A cache has at most 64 entries and a 12-window TTL. The exact signature includes demand, queue, capacity, buffer and output-port topology. Every reuse rebuilds current flow parameters and revalidates them. Healthy traffic uses a local no-op even if a previous overload left a queue. Reflection uses the measured outcome locally, avoiding a second serial API call.

Requests not admitted by pacing start no transport and are counted as deferred. A timed-out request is counted even though no response was received. Killing the local client cannot undo generation already received by a cloud server. No background API work is silently excluded from control timings. This is bounded application waiting on a normal OS, not a hard real-time guarantee.

# 6 Evaluation protocol and reproducibility

| Evidence | Design | Interpretation |
|---|---|---|
| Primary v4 | Seeds 1900–1909; 8 × 10 × 120 × 9 = 86,400 windows | Main offline network-policy comparison |
| Development | Seeds 1880 and 1881 | Parameter gates fixed before primary run |
| Final provider pilot | Seeds 1980 and 1981; 12 × 2 × 2 modes per provider | Actual requests, timed five-second observations |
| Accelerated diagnostic | Seeds 1920 and 1921 | Request admission under accelerated replay |
| Initial scheduled diagnostic | Seeds 1940 and 1941 | Revealed avoidable healthy/fallback repeat calls |
| Offline rule-learning demo | Seed 1880, 120 recurring windows | Shows exploration, evidence and rule lifecycle |
| Free-model availability | Every discovered zero-price model ID | Healthy schema availability, not quality |

All primary and diagnostic evidence is retained separately. The final timing rerun followed a cache/bypass optimization motivated by the first scheduled diagnostic; it uses new seeds. No primary network-policy parameters were tuned on held-out outcomes. A post-run audit repaired the binary rounding edge at the 20% floor and reran the full suite; pre-repair CSV and summary are preserved separately. The frozen values, trace hashes, runtime prefix and timestamps are recorded in summary JSON.

Seed means are the inference unit. Paired differences use a fixed-RNG, 4,000-draw percentile bootstrap with 95% intervals. Comparisons are exploratory without multiplicity correction. Provider pilots have only two seeds; individual windows are not independent seed replications. Mode order reverses for the second seed, while time-varying provider service remains a confound.

| ID | Role |
|---|---|
| A0 | Static primary |
| B1 | Least loaded |
| A2 | Original embedding |
| H1 | Guarded fixed 20% meter |
| A3 | Guarded balanced memory |
| V3 | Previous A4 (v3) |
| H2 | Capacity-aware meter, no memory |
| A4 | Loss-aware parameter memory |
| O2 | Privileged matched-action reference |

# 7 Primary benchmark: all controllers

| ID | Delay ms | Goodput Mbps | Mean loss % | Reward |
|---|---|---|---|---|
| A0 | 25.904 | 12.872 | 8.149 | 0.000000 |
| B1 | 20.609 | 13.024 | 6.996 | 0.061813 |
| A2 | 17.495 | 12.821 | 8.304 | 0.055081 |
| H1 | 17.131 | 12.956 | 7.391 | 0.077639 |
| A3 | 17.259 | 12.945 | 7.477 | 0.074975 |
| V3 | 17.131 | 12.956 | 7.391 | 0.077639 |
| H2 | 17.363 | 13.011 | 7.077 | 0.083304 |
| A4 | 17.452 | 13.011 | 7.075 | 0.082764 |
| O2 | 17.096 | 13.016 | 7.050 | 0.086096 |

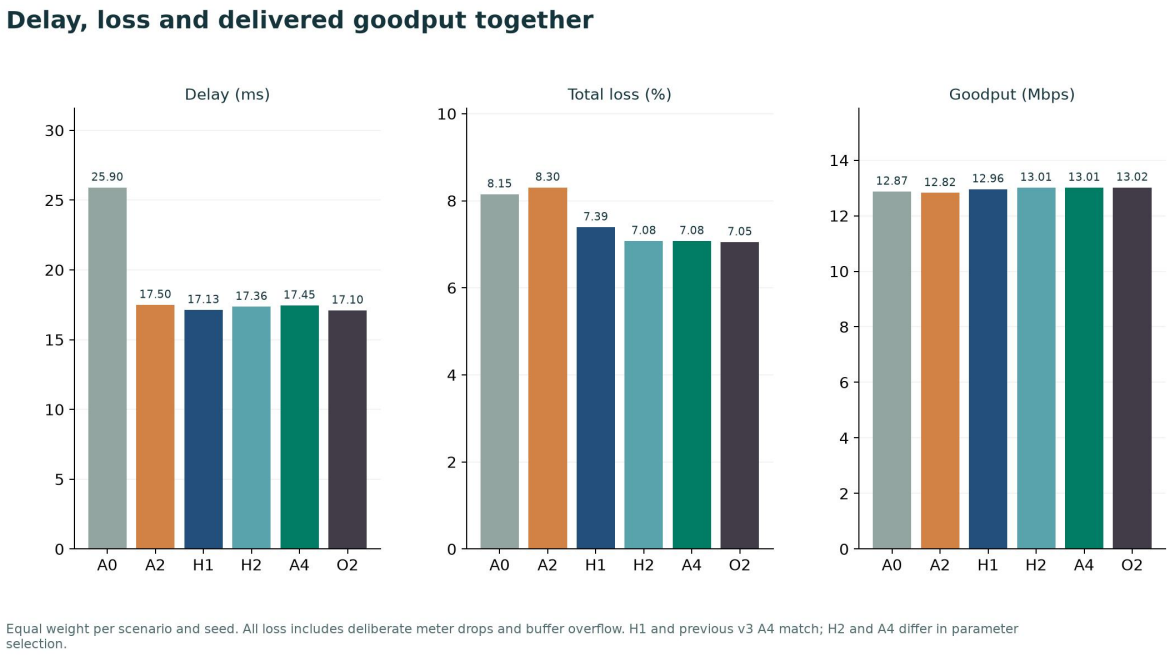



**Figure 3. Delay, total loss and delivered goodput on matched demand.**

A4 versus H1 loss difference was -0.316287 [-0.330606, -0.303954] percentage points. Reward difference was 0.005126 [0.004692, 0.005622]. Modeled delay difference was 0.321579 [0.280169, 0.367272] ms. Brackets are paired 95% seed intervals.

A4 had 0 safety vetoes. H1 and preserved previous A4 (V3) match in network behavior on this suite. Better reward and loss over H1 coexist with a small delay increase; lower loss is not a claim that every metric or scenario improves.

# 8 Scenario effects against H1

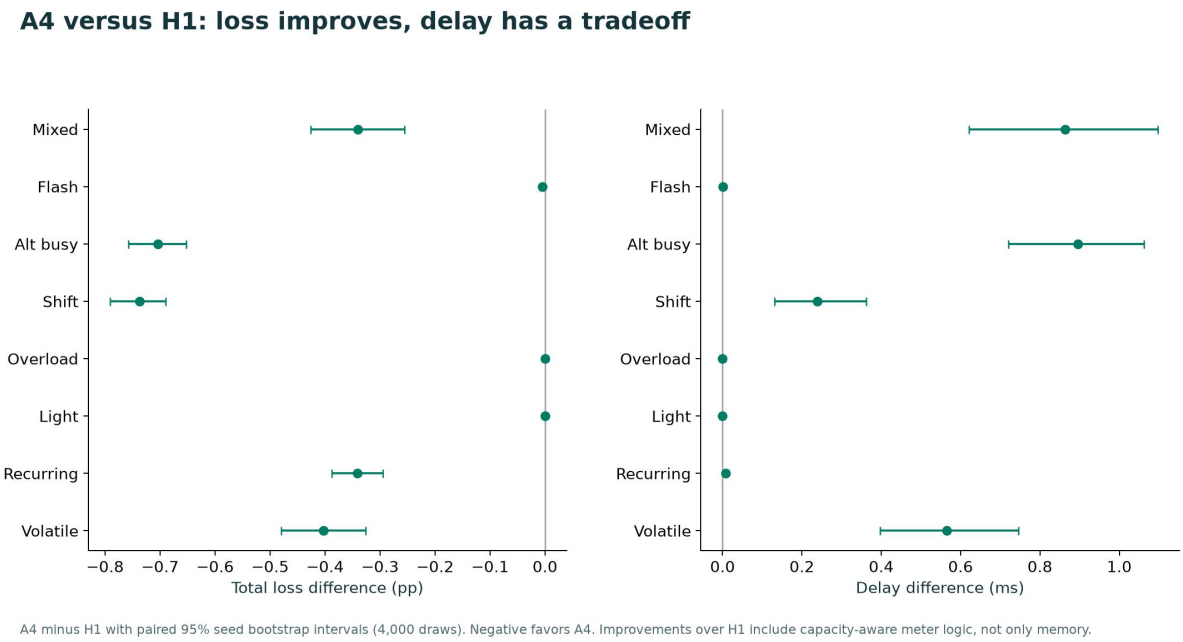



**Figure 4. A4 minus H1, paired seed uncertainty.**

| Scenario | Loss difference pp | Delay difference ms |
|---|---|---|
| Repeated mixed load | -0.341 | 0.864 |
| Primary flash crowd | -0.005 | 0.000 |
| Busy alternate path | -0.704 | 0.896 |
| Traffic distribution shift | -0.737 | 0.239 |
| Persistent dual overload | 0.000 | 0.000 |
| Light-load control | 0.000 | 0.000 |
| Recurring traffic contexts | -0.341 | 0.008 |
| Unpredictable traffic changes | -0.403 | 0.566 |

Traffic that already fits capacity needs no intervention. When the safety floor binds during heavy overload, meter candidates can become identical. Improvements are therefore context dependent. Scenario and pooled statistics are both preserved; no favorable subset replaces the complete primary result.

# 9 Matched no-memory ablation

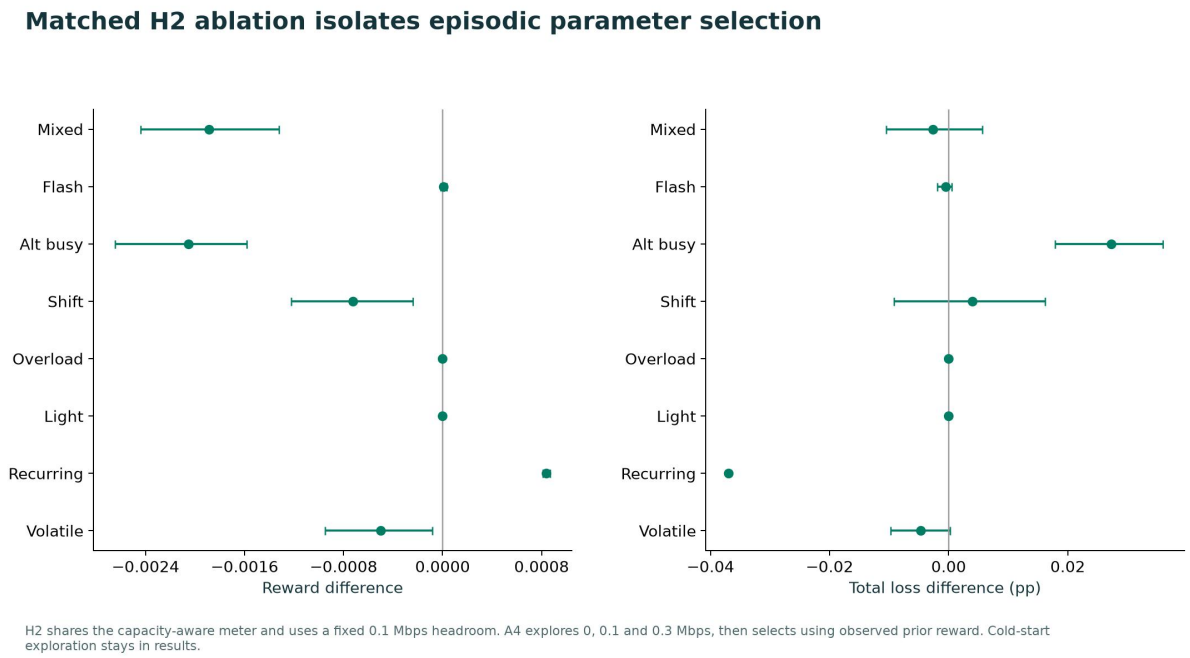



**Figure 5. A4 minus H2: the extra effect of episodic parameter selection.**

H2 reward was 0.083304, A4 reward 0.082764; paired A4 minus H2 -0.000540 [-0.000705, -0.000398]. A4 minus H2 loss was -0.001693 [-0.004857, 0.001384] pp and delay was 0.089460 [0.071521, 0.112085] ms. The primary suite does not establish a reward gain from adding memory to the new meter policy.

Memory retains adverse and positive experience, distinguishes meter parameters and demonstrably changes later parameter selection. That mechanism is different from establishing general performance superiority. Cold-start exploration can cost reward; keeping those windows avoids overstating learning benefit. H2 is the relevant attribution control, while H1 measures improvement over the previous baseline.

The privileged O2 reference uses the same action and headroom set but selects with same-window modeled outcomes. It is useful as a local comparison, not a deployable controller, causal estimate or long-horizon upper bound.

# 10 Actual Gemini and OpenRouter control timing

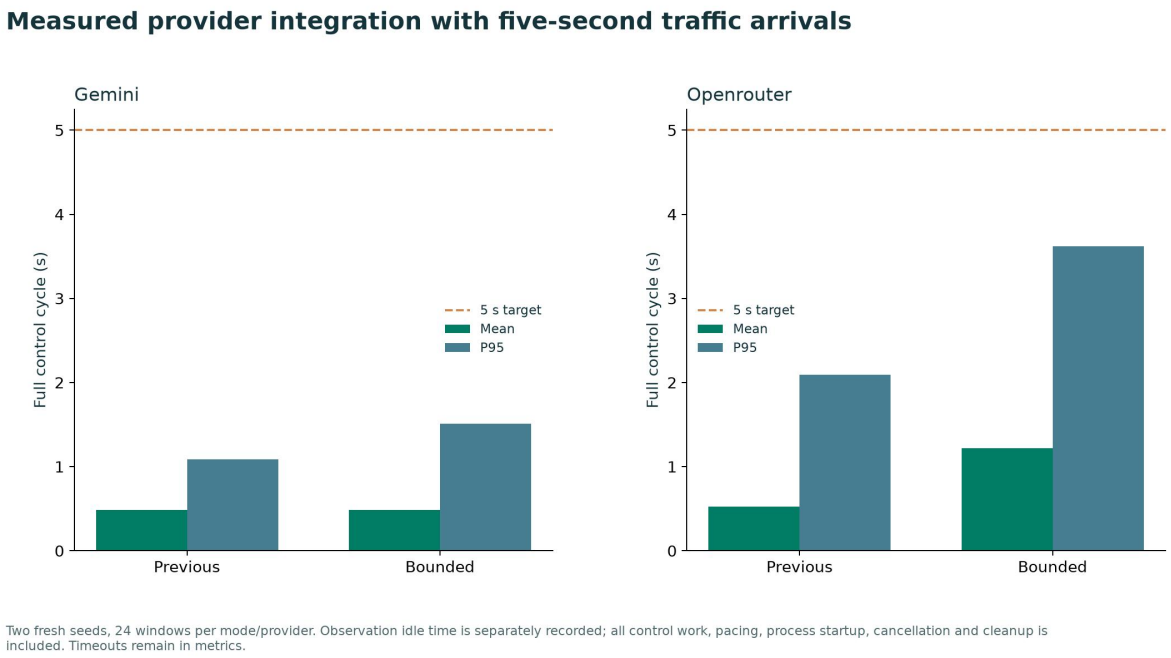



**Figure 6. Final timed traffic pilot; full-cycle mean and P95.**

| Provider / mode | Mean s | P95 s | Max s | 5 s misses |
|---|---|---|---|---|
| gemini / previous | 0.486 | 1.087 | 2.482 | 0/24 |
| gemini / bounded | 0.487 | 1.511 | 1.765 | 0/24 |
| openrouter / previous | 0.524 | 2.094 | 2.466 | 0/24 |
| openrouter / bounded | 1.222 | 3.620 | 3.621 | 0/24 |

The final pilot delivers observations every five wall-clock seconds. Idle observation wait is recorded separately; it is not provider work. Measured control includes actual waiting, admission pacing, startup, validation, outcome computation, rule expiry, reflection, memory update and worker cancellation. The earlier accelerated replay remains available to show request-admission pressure.

A lower P95 or fewer deadline misses can coexist with unchanged or higher mean latency when process isolation and fallbacks alter the mix of calls. These are system-level measurements, not proof that provider servers became faster. Exact mode and provider figures above govern the conclusion.

# 11 Provider coverage and network quality

| Provider / mode | Answered / calls | Timeouts | Deferred | Loss % | Reward |
|---|---|---|---|---|---|
| gemini / previous | 12/12 | 0 | 0 | 7.167 | 0.142863 |
| gemini / bounded | 10/10 | 0 | 0 | 6.975 | 0.147503 |
| openrouter / previous | 3/8 | 0 | 0 | 7.167 | 0.142863 |
| openrouter / bounded | 5/10 | 5 | 0 | 6.975 | 0.147503 |

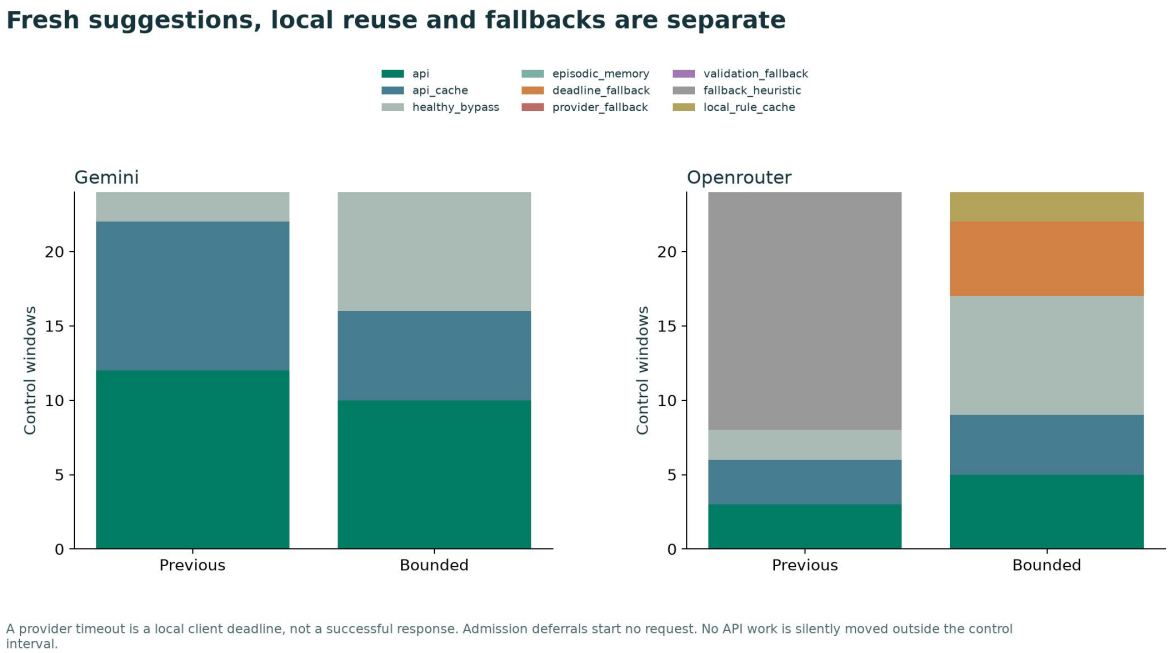



**Figure 7. Each window has an explicit decision origin.**

Success counts refer to schema-valid responses, including responses later rejected by the independent rule validator. API-action windows can therefore be fewer than successful responses. Provider failure, malformed output, timeout, local cache and local memory are never counted as fresh successful model decisions.

The compact schema requests an action and a short reason. Current congestion and alternate feasibility flags are computed locally. The provider does not choose arbitrary match fields or meter IDs. This benchmark evaluates the integrated pipeline on a small workload; it does not isolate a general causal benefit of LLM reasoning.

# 12 Complete rule-change and suggestion demo

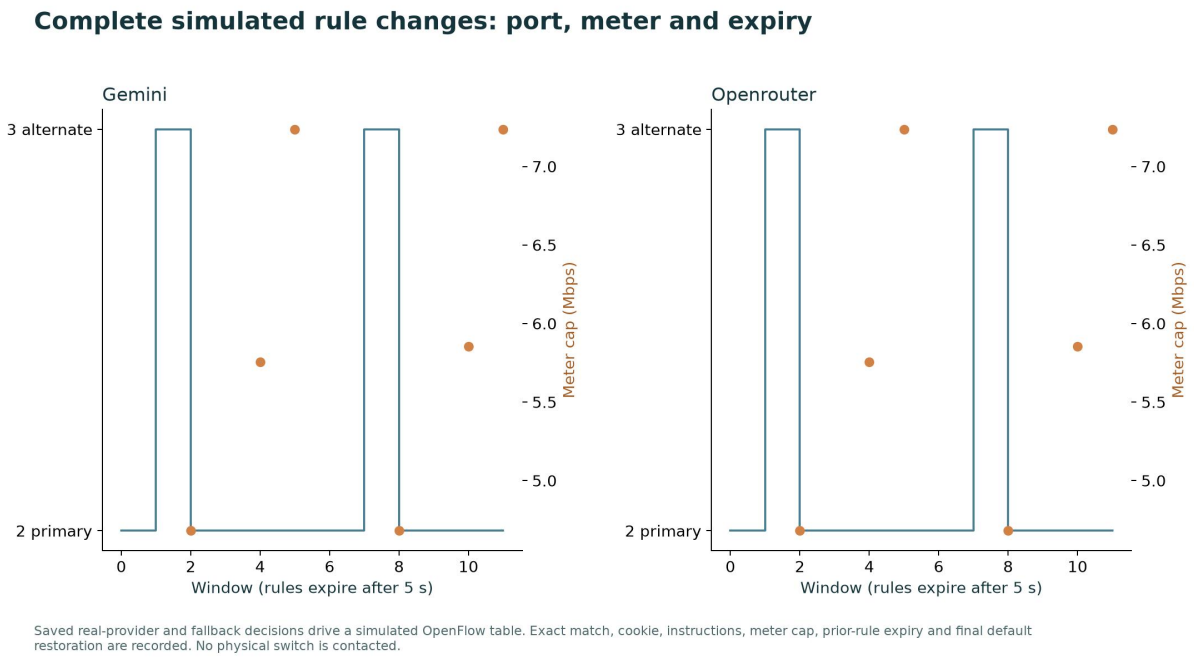



**Figure 8. Effective output port and meter cap from saved provider evidence.**

Open the Rule demo panel on the dashboard. Select Gemini or OpenRouter, choose a seed, then step or play the saved windows. The panel displays current demand and queues, the exact model suggestion, the independent guard result, the installed rule and expiry, the no-op comparison, measured outcome and memory evidence. Replay makes no new API calls.

A reroute is a simulated ADD rule for switch 1 and the exact IPv4 flow, priority 100, hard_timeout 5 seconds and OUTPUT port 3. A meter uses a METER instruction plus OUTPUT port 2. At the next virtual window the previous rule expires; a healthy window installs no intervention and default port 2 is used. A final event expires the last rule and confirms an empty intervention table.

Provider suggestions and installed decisions remain separate. In the first scheduled diagnostic, Gemini suggested a reroute into an alternate path at about 140.9% utilization; the validator rejected it and installed the local bounded meter. The raw suggestion, guard reason and installed outcome are preserved. A provider timeout similarly produces an explicitly labeled local fallback, never an invented answer.

For memory learning, select the separate 120-window offline replay and inspect later repeated contexts. It records parameter evidence, earlier cycle IDs, exploration and supported selection. It demonstrates policy mechanics; it does not replace the held-out performance comparison.

In addition to the Web Rule Demo, an interactive Tkinter Network Operations Console (selfx/live/console.py and selfx/live/engine.py) allows step-by-step visual stepping through the fluid model, displaying real-time packet flow animations, queue shifts, controller decisions, and counterfactual delay comparisons. The engine is verified by automated replay against saved v4 benchmark traces (test_live_engine.py).

# 13 Availability, runtime and validation

The new catalog sweep checked every discovered zero-price OpenRouter model: 1/17 returned a valid healthy-network compact action. Fourteen checks returned HTTP 429 (rate/quota limit) and two returned HTTP 403 (access denied); those failures remain in the evidence. Failed or unavailable checks are retained with sanitized audit codes. Model availability changes over time; the record is a timestamped integration check, not a ranking or guarantee of free quota.

Gemini uses the configured Gemini 3.5 Flash-Lite model; actual returned versions and token usage are preserved. OpenRouter control uses only the free router or model IDs ending in :free. Keys remain in the ignored .env file and are excluded from documents, source archives and logs. The isolated Python environment is .venv; .env is a configuration file, not a virtual-environment directory.

The complete portable project is executed in .venv, including all five normal-runner controllers. Verification checks actual data against summary means, paired intervals, trace hashes, strict past-only retrieval, safety, traffic-volume conservation, rule expiry and provider counts. A hung-worker test confirms bounded return and no surviving local child process; other tests cover pacing admission, honest fallback caching and healthy bypass.

Browser validation covers both provider replays, the learning replay, navigation, benchmark selection, an offline run, desktop/mobile rendering, absence of JavaScript errors, graph downloads and all editable/PDF downloads. Final DOCX and PPTX are exported by native Word and PowerPoint and visually reviewed page by page. PowerPoint charts and tables stay editable, with embedded chart workbooks.

The exact test result and final verification status are in output/runtime_validation.json and output/verification.json. Earlier v2/v3 raw evidence and previous deliverables are preserved separately; canonical submission files contain this revision only.

# 14 Run, reproduce and edit

.\setup.ps1

.\run.ps1 all

.\run.ps1 dashboard -Port 8766

.\.venv\Scripts\python.exe -m eval.run_v4_latency --provider gemini --realtime --seeds 1980 1981 --out-dir output/benchmarks/v4/latency_gemini_pooled

.\.venv\Scripts\python.exe -m eval.run_v4_latency --provider openrouter --realtime --seeds 1980 1981 --out-dir output/benchmarks/v4/latency_openrouter_pooled

.\.venv\Scripts\python.exe -m eval.build_rule_demo

Open http://127.0.0.1:8766 locally. Live commands consume available quota; use a distinct output directory to preserve prior runs. The saved replay and offline comparison need no provider request. Requirements, setup and run scripts are local to selfx.

| Artifact | Location |
|---|---|
| Editable Word report / exported PDF | output/submission/SELF_X_Report.docx and .pdf |
| Editable presentation / exported PDF | output/submission/SELF_X_Presentation.pptx and .pdf |
| All graph PNGs and SVGs | output/benchmarks/v4/figures/ |
| 17-page graph atlas | output/submission/SELF_X_Benchmark_Atlas.pdf |
| Primary CSV, summaries and hashed traces | output/benchmarks/v4/ |
| Provider suggestions, rule events and expiries | latency_gemini_pooled/ and latency_openrouter_pooled/ |
| Reproduction sources and evidence ZIP | output/submission/SELF_X_Submission.zip |

Authoring: scripts/build_v4_artifacts.py rebuilds PNG/SVG graphs, the atlas, chart data and editable report. scripts/build_v4_presentation.mjs builds native slides in Artifact Session; finalize_v4_presentation.mjs validates geometry and embeds editable chart workbooks. DOCX/PPTX PDFs are exported from the actual native files. The graph manifest records source hashes.

Interface Distinction: The repository provides two distinct graphical environments: (1) Public Dashboard & API (python Web/server.py on http://localhost:5000), presenting public read-only result APIs and cross-controller comparisons; and (2) Offline Review Dashboard (selfx/run.ps1 dashboard on http://127.0.0.1:8766) and Tkinter Live Console (.\run.ps1 simulator) for granular window-by-window replay.

# 15 Limitations and references

This is a synthetic two-path constant-rate fluid model. It does not model TCP, packet scheduling, packet-level flow export, real SDN installation latency or real switch behavior. Traffic is generated locally and is not calibrated to a captured external dataset. The reward uses a modeled same-window no-op reference; this counterfactual is a measurement aid, not an additional observation available in a real deployment.

The memory uses engineered numeric context and observed outcomes, not trained embedding weights or LLM fine-tuning. A4 improves loss/reward over H1 while increasing modeled delay. It does not establish a reward gain over matched H2. The two-seed API pilot and changing provider service limit generalization. A local deadline bounds client waiting on this machine; cloud generation may continue after cancellation, and no production SLA is established.

External approaches are represented by a least-loaded path baseline, a guarded ECMP-style baseline in preserved v3 evidence, and privileged model-based references. Literature discusses concepts and mechanisms; its reported numbers are not mixed with these synthetic results or claimed as directly comparable benchmark measurements.

Shinn et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS. https://arxiv.org/abs/2303.11366

Rusek et al. (2019). RouteNet: Leveraging Graph Neural Networks for network modeling and optimization in SDN. https://arxiv.org/abs/1910.01508

Google. Gemini API model catalog. https://ai.google.dev/gemini-api/docs/models

Google. Structured outputs. https://ai.google.dev/gemini-api/docs/structured-output

Google. Gemini API rate limits. https://ai.google.dev/gemini-api/docs/rate-limits

OpenRouter. Free Models Router. https://openrouter.ai/docs/guides/routing/routers/free-router

Ryu. ofctl_rest reference. https://ryu.readthedocs.io/en/latest/app/ofctl_rest.html

IETF RFC 2992. Analysis of an Equal-Cost Multi-Path Algorithm. https://www.rfc-editor.org/rfc/rfc2992

IETF RFC 7011. Specification of the IP Flow Information Export (IPFIX) Protocol. https://www.rfc-editor.org/rfc/rfc7011

# 16 Team Contributions and Division of Work

This project was developed collaboratively by Team SDN_07 for EC466: Software Defined Networks at IIIT Dharwad. To maintain rigorous academic accountability, each member's verified contributions across engineering, evaluation infrastructure, experimental integration, interface development, and testing are documented below.

| Team Member | Role & Responsibilities | Key Modules & Deliverables | Verified Commits & Tests |
|---|---|---|---|
| M Jagadeeswar | Benchmark Evaluation Infrastructure, File-Based Result Persistence, Backend APIs & Data Integrity | selfx/eval/result_schema.py, result_writer.py, result_integration.py, run_ablation.py; Web/server.py, Web/js/results.js, Web/js/charts.js | Commits: 37048ea, 0d730db, cbe082f, c86708a; Tests: test_result_schema, test_result_writer, test_result_integration, test_public_api, test_results_integration |
| J Ganesh Kumar Reddy | Core Agent Architecture, Benchmark Suite & Evaluation Pipelines | selfx/agent/ (loss_aware_memory, bounded_control, safety), selfx/eval/run_v4_benchmarks.py, scripts/build_v4_artifacts.py, v4_visuals.py | Commits: e4d5bb2, bfa76f0, 54e4e1e, 55226da, fadaaca, 89c33aa; 86,400-window v4 evaluation, test_v4_control, test_simulation |
| Harsh Raj | Web Dashboard UI Architecture, Visual Styling & Theme Optimization | Web/CSS/Style.css, Web/index.html, Web/js/app.js, Web/js/topology.js | Commits: 715ec3d, ab654b6 series (Oct 11), 9ce2271; Dashboard layout, light/dark theme contrast synchronization, toast notifications |
| comet (ft. Aravind & Koushik) | Slow-Loop Policy Governor Prototype & Tkinter Live Operations Console | slow-loop/ (governor, agents, benchmark, tests), selfx/live/console.py, selfx/live/engine.py, scripts/explainer_figures.py | Commits: bf873fd, 27cbd5e, 213bb45; Standalone two-timescale governor prototype, live GUI simulator console, test_live_engine |

## 16.1 M Jagadeeswar — Benchmark Evaluation Infrastructure, File-Based Result Persistence, Backend API Development & Data Integrity

M Jagadeeswar's core contributions established the end-to-end evaluation data pipeline that bridges raw controller execution with verifiable, reproducible academic evidence. Rather than merely presenting numbers on a frontend, his work engineered the backend schemas, file-based result persistence, public REST APIs, and data integrity safeguards that make the project's experimental findings structured, auditable, and reliable:

**Engineering Pipeline: Experiment Execution → Structured Result Schema → Validation and Atomic JSON Persistence → Ablation Pipeline Integration → Result Aggregation → Read-Only REST APIs → Dynamic Dashboard Integration → Data Integrity and Automated Testing.**

### 16.1.1 Structured Evaluation Infrastructure and File-Based Result Persistence

To resolve the absence of a standardized result format across experimental stages, Jagadeeswar designed and implemented the structured experiment schema in selfx/eval/result_schema.py. This module defines type-safe models (RunMetadata, ScenarioResult, ControllerMetrics, and AblationOutput) enforcing mandatory fields, metric unit consistency, and confidence interval structures. He also

authored selfx/eval/result_writer.py, implementing validated, atomic JSON serialization to disk with cryptographic checksum generation. This file-based persistence model avoids heavyweight database dependencies while guaranteeing that multi-seed benchmark runs produce consistent, auditable, and immutable JSON artifacts on disk. (Verified in commit 37048ea).

#### 16.1.2 Ablation Evaluation and Result Integration

In selfx/eval/run_ablation.py, Jagadeeswar integrated the evaluation harness with the structured result writer, enabling automated collection of baseline (A0), heuristic (A1), and guard/memory (A2) runs under identical schema constraints. He authored selfx/eval/result_integration.py to aggregate multi-condition evaluation outputs into uniform payloads for analytical consumption, decoupling experiment execution from visualization. (Verified in commit 37048ea).

#### 16.1.3 Public Read-Only Benchmark APIs

In Web/server.py, Jagadeeswar engineered public, read-only REST API endpoints (/api/results, /api/results/ablation, and /api/results/meta). These backend endpoints retrieve persisted experiment artifacts from disk and serve structured summaries, scenario-specific ablation matrices, and execution metadata programmatically. This backend development provides secure, decoupled, and read-only academic endpoints that allow external evaluators to inspect benchmark outputs without frontend coupling. (Verified in commit 0d730db).

#### 16.1.4 Benchmark Dashboard Integration and Visualization

Jagadeeswar authored Web/js/results.js (over 750 lines), connecting the backend read-only APIs to the Web dashboard interface. His implementation parses raw benchmark payloads, computes client-side aggregations, dynamically renders controller comparison tables, and feeds validated data into Web/js/charts.js for interactive trade-off visualization. Clearly distinguished from Harsh Raj's foundational dashboard UI architecture and visual styling, Jagadeeswar's contribution focused specifically on benchmark result ingestion, dynamic data aggregation, and analytical visualization. (Verified in commits cbe082f and c86708a).

#### 16.1.5 Data Integrity, Result Reliability and Validation

A critical aspect of Jagadeeswar's work was enforcing scientific data integrity across the project deliverables and dashboards. He conducted an extensive correctness audit: (1) completely removed misleading hardcoded KPI values, fabricated percentage deltas, and synthetic sparklines; (2) implemented strict missing-value handling so absent confidence intervals or veto counts display honest '—' or 'N/A' indicators instead of zero substitution or false error bars; (3) enforced explicit UI badges and metadata distinguishing offline fluid-model benchmarks, synthetic browser illustrations, external LLM API diagnostics, and live switch telemetry; and (4) authored LLM diagnostic disclaimers ensuring API time-budget sweeps are not misrepresented as autonomous packet-level forwarding. (Verified in commit c86708a).

#### 16.1.6 Testing and Verification

Jagadeeswar backed every module with dedicated automated test suites: selfx/eval/tests/test_result_schema.py (schema validation and constraint checks), selfx/eval/tests/test_result_writer.py (atomic writing and directory handling), selfx/eval/tests/test_result_integration.py (payload normalization), Web/tests/test_public_api.py (endpoint status codes, headers, and read-only contracts), and Web/tests/test_results_integration.py (client-side data rendering and error handling). All tests pass deterministically in the local test environment. (Verified in commits 37048ea, 0d730db, cbe082f, c86708a).

### 16.2 J Ganesh Kumar Reddy — Core Agent Architecture, Benchmark Suite & Evaluation Pipelines

J Ganesh Kumar Reddy initialized the repository foundation, configured the initial Ryu controller and Mininet two-path topology scripts (ryu_multipath.py, topo_2path.py), and authored the core agent modules in selfx/agent/ (including loss_aware_memory.py, bounded_control.py, and safety.py). He developed the comprehensive 86,400-window v4 benchmark evaluation suite

(selfx/eval/run_v4_benchmarks.py), authored artifact and figure generation pipelines (scripts/build_v4_artifacts.py, scripts/v4_visuals.py), and created the submission verification script (scripts/verify_v4_submission.py) enforcing strict volume conservation across simulated runs. (Verified in commits e4d5bb2, bfa76f0, 54e4e1e, 55226da, fadaaca, 89c33aa).

### 16.3 Harsh Raj — Web Dashboard UI Architecture, Visual Styling & Theme Optimization

Harsh Raj designed and implemented the foundational visual architecture of the public Web dashboard, authoring Web/CSS/Style.css, Web/index.html, and interactive client components in Web/js/app.js. In commit 9ce2271, he conducted substantial visual refinements: resolving light theme contrast deficiencies across terminal consoles and toast notifications, refining episodic memory card layouts, synchronizing topology link utilization metrics with KPI inspector panels, and implementing smooth responsive breakpoints across desktop and mobile displays. (Verified in commits 715ec3d, ab654b6 series, 9ce2271).

### 16.4 comet (ft. Aravind & Koushik) — Slow-Loop Policy Governor Prototype & Tkinter Live Operations Console

comet (co-developed with Aravind and Koushik in commit bf873fd) researched and developed the standalone slow-loop policy governor prototype under slow-loop/, investigating a two-timescale architecture where an AI governor tunes fast mathematical controller parameters once a minute. In commit 27cbd5e, comet developed the Tkinter-based Network Operations Console (selfx/live/console.py and selfx/live/engine.py), providing an interactive window-by-window visual simulation of packet dynamics, accompanied by explainer visualizations in selfx/output/explainer/ and verified by selfx/tests/test_live_engine.py. (Verified in commits bf873fd, 27cbd5e, 213bb45).